

[NOM DE L'ÉTABLISSEMENT]

Département Génie Logiciel / Informatique

RAPPORT DE PROJET

Application mobile de gestion des TPE

Gestion des terminaux de paiement électronique, des stations, des pannes et des interventions

Réalisé par : Chams Eddine Mahmoud

Encadrant : Nidhal Chebbi

Année universitaire : 2026–2027

Technologies : Spring Boot – Flutter – MySQL – JWT

Projet académique

Remerciements

La réalisation de ce projet constitue une étape importante dans notre apprentissage du développement logiciel. Nous souhaitons remercier l'ensemble des personnes qui ont contribué, directement ou indirectement, à sa réalisation.

Nous adressons tout d'abord nos remerciements à notre encadrant pour son accompagnement, ses remarques et ses conseils tout au long du projet. Ses observations ont permis d'améliorer progressivement la conception de la solution et de mieux comprendre les contraintes liées au développement d'une application complète.

Nous remercions également les enseignants et responsables pédagogiques pour les connaissances acquises durant notre formation. Les notions de bases de données, programmation orientée objet, génie logiciel, développement web et mobile ont constitué une base essentielle pour réaliser ce travail.

Enfin, nous remercions toutes les personnes ayant participé aux échanges, aux tests et aux retours sur l'application.

Résumé

Ce projet consiste à concevoir et développer une solution informatique destinée à la gestion des terminaux de paiement électronique (TPE). L'objectif principal est de centraliser les informations relatives aux TPE, aux stations auxquelles ils sont affectés, aux pannes rencontrées et aux interventions réalisées par les techniciens.

La solution repose sur une architecture séparant le backend et l'application mobile. Le backend est développé avec Spring Boot et expose une API REST. Les données sont persistées dans une base de données MySQL grâce à Spring Data JPA. L'accès à l'application est sécurisé à l'aide d'une authentification JWT et d'un système de rôles distinguant notamment les administrateurs et les techniciens.

L'application mobile est développée avec Flutter. Elle permet aux utilisateurs autorisés de consulter les TPE, effectuer des recherches, déclarer des pannes, suivre les interventions et consulter des indicateurs de fonctionnement. Les administrateurs disposent en outre de fonctionnalités de gestion des référentiels, des stations et des affectations.

Une attention particulière est accordée au cycle de vie d'un TPE. Les états permettent de représenter son évolution, depuis son stockage jusqu'à sa mise en service, sa réparation ou sa mise hors service.

Mots-clés : TPE, terminal de paiement, Spring Boot, Flutter, MySQL, API REST, JWT, JPA, gestion des pannes, intervention.

Table des matières

Remerciements	i
Résumé	ii
1 Introduction générale	1
1.1 Contexte	1
1.2 Problématique	1
1.3 Objectifs	2
1.4 Méthodologie	2
2 Présentation du projet	3
2.1 Présentation générale	3
2.2 Acteurs	3
2.2.1 Technicien	3
2.2.2 Administrateur	4
2.3 Périmètre fonctionnel	4
3 Analyse fonctionnelle	5
3.1 Cas d'utilisation	5
3.2 Diagramme de cas d'utilisation	6
3.3 Description des cas d'utilisation	6
3.3.1 S'authentifier	6
3.3.2 Consulter les TPE	7
3.3.3 Déclarer une panne	7
3.3.4 Créer une intervention	7
3.4 Règles de gestion	7
4 Conception des données	8
4.1 Identification des entités	8
4.2 Entité TPE	8
4.3 Entité Station	8

4.4	Entité Panne	8
4.5	Entité Intervention	9
4.6	MLD	9
4.7	Contraintes d'intégrité	9
5	Architecture technique	10
5.1	Architecture générale	10
5.2	Backend Spring Boot	10
5.3	Application Flutter	10
5.4	Communication REST	11
6	Mise en place de l'environnement	13
6.1	Préparation du poste	13
6.2	Projet Spring Boot	13
6.3	Configuration MySQL	13
6.4	Initialisation Flutter	14
6.5	Premier test d'intégration	14
7	Développement du backend	15
7.1	Création des entités	15
7.2	Repository	15
7.3	Service	16
7.4	Contrôleur REST	16
8	Authentification et sécurité	18
8.1	Objectif	18
8.2	JWT	18
8.3	Rôles	18
8.4	Bonnes pratiques	19
9	Gestion du cycle de vie des TPE	20
9.1	Principe	20
9.2	Affectation	20
9.3	Déclaration d'une panne	20
9.4	Réparation	20
9.5	Remise en service	20
9.6	Hors service	21
9.7	Implémentation des transitions	21

10 Gestion des pannes et interventions	22
10.1 Déclaration d'une panne	22
10.2 Référentiel des types de panne	22
10.3 Intervention	22
10.4 Historique	23
11 Développement de l'application Flutter	24
11.1 Objectif de l'interface	24
11.2 Écran de connexion	24
11.3 Liste des TPE	25
11.4 Détail d'un TPE	25
11.5 Ajout et modification	25
11.6 Gestion des stations	25
11.7 Pannes et interventions	25
12 Dashboard et statistiques	26
12.1 Objectif	26
12.2 Indicateurs	26
12.3 Statistiques avancées	26
12.4 Graphiques Flutter	27
13 API REST et documentation	28
13.1 Principes REST	28
13.2 Format JSON	28
13.3 Gestion des erreurs	28
13.4 Swagger / OpenAPI	29
14 Tests et validation	30
14.1 Stratégie	30
14.2 Scénario d'authentification	30
14.3 Scénario de panne	30
14.4 Tests unitaires	31
15 Déploiement	32
15.1 Architecture de déploiement	32
15.2 Environnement de développement	32
15.3 Environnement de production	32
15.4 Sécurité du déploiement	32

16 Organisation et planification	34
16.1 Semaine 1 : conception et setup	34
16.2 Semaine 2 : coeur backend	34
16.3 Semaine 3 : statistiques et Flutter	34
16.4 Semaine 4 : intégration et livraison	34
17 Difficultés et solutions	35
17.1 Complexité du cycle de vie	35
17.2 Communication mobile/backend	35
17.3 Authentification	35
17.4 Gestion des données	35
17.5 Gestion du temps	35
18 Bilan et perspectives	37
18.1 Bilan	37
18.2 Compétences acquises	37
18.3 Perspectives	38
Conclusion générale	39
A Annexe A – Arborescence Spring Boot	40
B Annexe B – Arborescence Flutter	41
C Annexe C – Exemple SQL	42
D Annexe D – Exemple de requêtes API	43
E Annexe E – Checklist de livraison	44

Table des figures

2.1	Vue globale de l'application à insérer	4
3.1	Diagramme UML des cas d'utilisation : Administrateur et Technicien .	6
5.1	Architecture générale : Flutter → API Spring Boot → MySQL	11
10.1	Écran d'historique d'un TPE à insérer	23
11.1	Capture de l'écran de connexion Flutter	24
11.2	Capture de la liste des TPE	25
12.1	Dashboard avec indicateurs et graphiques	27
12.2	stat	27
15.1	Diagramme de déploiement UML	33

Liste des tableaux

3.1 États principaux d'un TPE	7
5.1 Exemples d'endpoints	12
13.1 Codes HTTP utilisés	29
14.1 Exemple de plan de tests	31
16.1 Planning prévisionnel	34

Chapitre 1

Introduction générale

1.1 Contexte

La gestion d'un parc de terminaux de paiement électronique nécessite de suivre un nombre important d'équipements, leurs affectations, leur état de fonctionnement ainsi que les opérations de maintenance effectuées sur chacun d'entre eux.

Dans une organisation utilisant plusieurs stations, la simple conservation d'informations statiques sur les terminaux devient rapidement insuffisante. Lorsqu'un TPE tombe en panne, il est nécessaire d'identifier précisément le matériel concerné, connaître son historique, enregistrer la panne, suivre l'intervention et déterminer s'il peut être remis en service ou s'il doit être déclaré hors service.

Le présent projet répond à cette problématique en proposant une application centralisée accessible depuis une interface mobile.

1.2 Problématique

Sans système centralisé, plusieurs difficultés peuvent apparaître :

- difficulté à connaître l'état actuel d'un TPE ;
- perte ou dispersion des informations historiques ;
- difficulté à suivre les pannes et les interventions ;
- absence de visibilité sur les équipements les plus souvent concernés par des incidents ;
- risque d'erreurs lors de l'affectation d'un TPE à une station ;
- difficulté à contrôler les droits des différents utilisateurs.

La problématique principale peut donc être formulée ainsi :

Comment concevoir une application sécurisée permettant de centraliser et de suivre le cycle de vie des TPE, leurs affectations, leurs pannes et leurs interventions, tout en fournissant aux différents acteurs les fonctionnalités correspondant à leurs responsabilités ?

1.3 Objectifs

L'objectif général est de développer une application mobile connectée à une API REST permettant une gestion structurée du parc de TPE.

Les objectifs spécifiques sont :

1. authentifier les utilisateurs ;
2. distinguer les rôles administrateur et technicien ;
3. gérer les TPE et les stations ;
4. gérer les affectations ;
5. suivre les états des TPE ;
6. déclarer et suivre les pannes ;
7. enregistrer les interventions ;
8. conserver un historique exploitable ;
9. fournir des recherches et statistiques ;
10. proposer une interface mobile simple et utilisable.

1.4 Méthodologie

Le projet est réalisé progressivement selon une approche itérative. La première phase est consacrée à l'analyse et à la conception. La deuxième concerne la mise en place du backend et de la base de données. La troisième porte sur le développement de l'application Flutter. La dernière phase est consacrée à l'intégration, aux tests et à la documentation.

Chapitre 2

Présentation du projet

2.1 Présentation générale

L'application proposée est un système de gestion des terminaux de paiement électronique. Elle met en relation plusieurs éléments : utilisateurs, TPE, stations, pannes, types de panne, interventions et types d'intervention.

Le système est construit autour d'une API REST. L'application Flutter joue le rôle de client tandis que Spring Boot assure la logique métier, la sécurité et l'accès aux données.

2.2 Acteurs

Deux acteurs principaux sont identifiés.

2.2.1 Technicien

Le technicien est principalement responsable du suivi opérationnel des TPE. Il peut :

- s'authentifier ;
- consulter les TPE ;
- rechercher un TPE ;
- consulter l'historique ;
- déclarer une panne ;
- créer une intervention ;
- enregistrer un diagnostic ou une réparation ;
- remettre un TPE en service ;
- consulter le tableau de bord.



FIGURE 2.1 – Vue globale de l'application à insérer

2.2.2 Administrateur

L'administrateur dispose des droits du technicien et de fonctionnalités supplémentaires :

- ajouter, modifier et supprimer des TPE ;
- gérer les stations ;
- affecter ou désaffecter un TPE ;
- gérer les types de panne ;
- gérer les types d'intervention ;
- consulter des statistiques avancées.

2.3 Périmètre fonctionnel

Le périmètre couvre la gestion du cycle de vie des TPE, la maintenance et la gestion des utilisateurs. Les fonctions qui ne sont pas indispensables au fonctionnement central, telles que certaines visualisations avancées, peuvent être considérées comme des extensions.

Chapitre 3

Analyse fonctionnelle

3.1 Cas d'utilisation

Les principaux cas d'utilisation sont :

- S'authentifier ;
- Consulter les TPE ;
- Rechercher un TPE ;
- Consulter l'historique ;
- Déclarer une panne ;
- Créer une intervention ;
- Remettre un TPE en service ;
- Consulter le tableau de bord ;
- Gérer les TPE ;
- Gérer les stations ;
- Affecter un TPE ;
- Gérer les référentiels.

3.2 Diagramme de cas d'utilisation

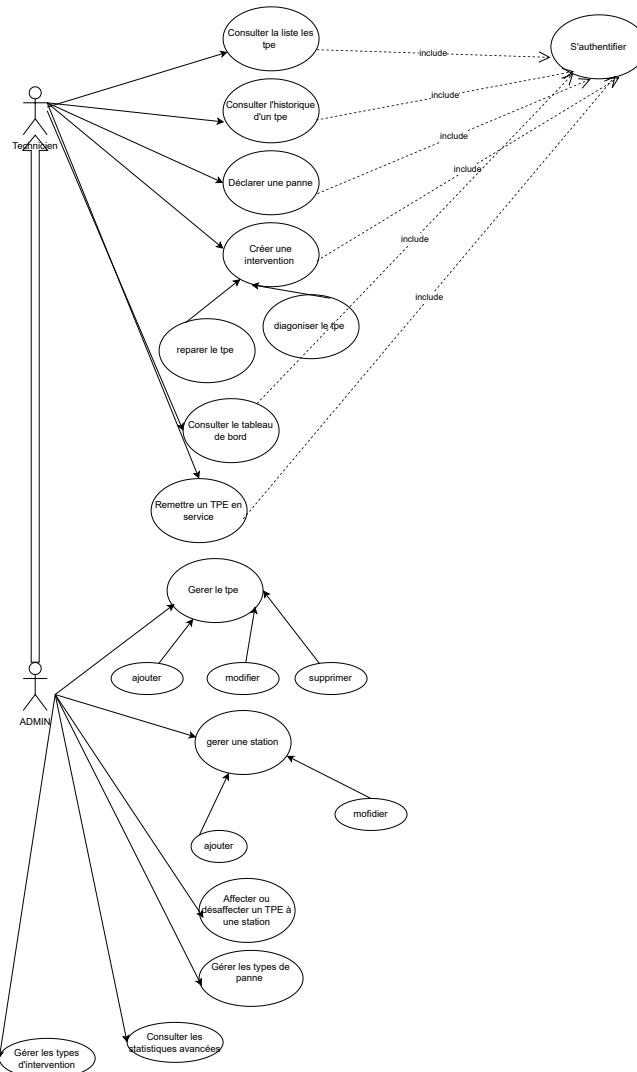


FIGURE 3.1 – Diagramme UML des cas d'utilisation : Administrateur et Technicien

3.3 Description des cas d'utilisation

3.3.1 S'authentifier

L'utilisateur saisit son identifiant et son mot de passe. Le backend vérifie les informations et retourne un jeton JWT lorsque l'authentification est valide. Le rôle de l'utilisateur est également pris en compte pour déterminer les fonctionnalités accessibles.

3.3.2 Consulter les TPE

Le technicien peut afficher la liste des TPE enregistrés. Pour chaque terminal, des informations telles que le numéro de série, l'état et la station peuvent être affichées.

3.3.3 Déclarer une panne

Une panne est associée à un TPE. Sa déclaration entraîne une évolution de l'état du terminal vers un état correspondant à une panne. Le système conserve les informations nécessaires pour permettre le suivi ultérieur.

3.3.4 Créer une intervention

Une intervention est associée à une panne. Elle contient notamment un type, un diagnostic, une description et les informations relatives à la réparation.

3.4 Règles de gestion

Le système doit empêcher les incohérences dans le cycle de vie. Par exemple, un TPE déjà hors service ne doit pas être remis en service comme s'il était simplement en réparation.

Les transitions principales sont représentées par :

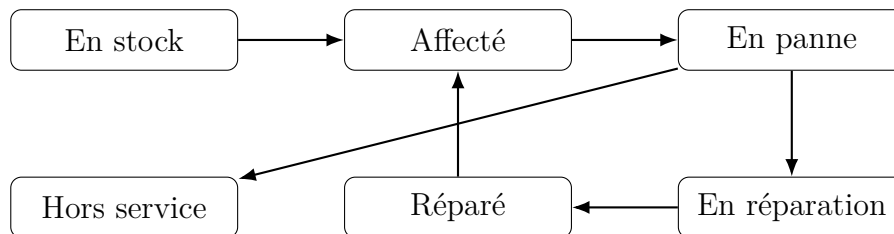


TABLE 3.1 – États principaux d'un TPE

État	Description
En stock	TPE disponible mais non affecté.
Affecté	TPE associé à une station.
En panne	TPE présentant un incident déclaré.
En réparation	Intervention de maintenance en cours.
Réparé	Intervention terminée avec possibilité de remise en service.
Hors service	TPE qui ne doit plus être utilisé.

Chapitre 4

Conception des données

4.1 Identification des entités

Les principales entités sont :

- Utilisateur ;
- TPE ;
- Station ;
- Panne ;
- TypePanne ;
- Intervention ;
- TypeIntervention.

4.2 Entité TPE

Le TPE représente le terminal de paiement. Il possède un identifiant technique, un numéro de série, un état et éventuellement une station d'affectation.

4.3 Entité Station

Une station représente le lieu ou l'unité opérationnelle à laquelle plusieurs TPE peuvent être associés.

4.4 Entité Panne

Une panne représente un incident rencontré sur un TPE. Elle est liée à un type de panne et permet d'initier un processus d'intervention.

4.5 Entité Intervention

Une intervention représente une action de maintenance. Elle peut contenir un diagnostic, une description de la réparation, une date de début, une date de fin et un statut.

4.6 MLD

Le passage au modèle logique permet de transformer les associations du MCD en relations adaptées au SGBD relationnel.

Exemple de relations :

- UTILISATEUR(id, nom, email, motDePasse, role) ;
- STATION(id, nom, adresse) ;
- TPE(id, numeroSerie, etat, station_id) ;
- TYPE_PANNE(id, libelle, description) ;
- PANNE(id, dateDeclaration, description, tpe_id, typePanne_id) ;
- TYPE_INTERVENTION(id, libelle, description) ;
- INTERVENTION(id, dateDebut, dateFin, diagnostic, description, panne_id, typeIntervention_id, technicien_id).

4.7 Contraintes d'intégrité

Les contraintes importantes sont :

- le numéro de série d'un TPE doit être unique ;
- un TPE ne peut être affecté qu'à une station valide ;
- une panne doit concerner un TPE existant ;
- une intervention doit être associée à une panne ;
- un utilisateur possède un rôle valide ;
- les transitions d'état doivent respecter les règles métier.

Chapitre 5

Architecture technique

5.1 Architecture générale

L'application suit une architecture en trois grandes parties :

1. une application mobile Flutter ;
2. une API backend Spring Boot ;
3. une base de données MySQL.

La communication entre Flutter et Spring Boot est réalisée au moyen d'une API REST utilisant principalement le format JSON.

5.2 Backend Spring Boot

Le backend est organisé selon plusieurs couches :

- `controller` : exposition des endpoints REST ;
- `service` : logique métier ;
- `repository` : accès aux données ;
- `model/entity` : représentation des entités ;
- `security` : authentification et autorisation ;
- `dto` : objets utilisés pour les échanges API.

5.3 Application Flutter

Flutter est utilisé pour construire l'interface mobile. L'application peut être organisée par fonctionnalités afin de faciliter son évolution.

Une organisation possible est :

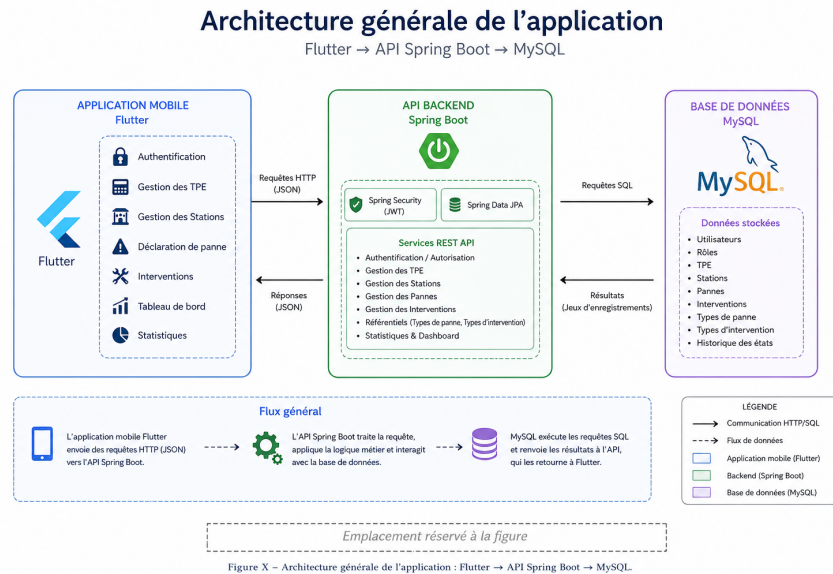


FIGURE 5.1 – Architecture générale : Flutter → API Spring Boot → MySQL

Listing 5.1 – Structure simplifiée Flutter

```

1 lib/
2   core/
3     constants/
4     network/
5     storage/
6   models/
7   services/
8   providers/
9   screens/
10  login/
11  tpe/
12  station/
13  panne/
14  intervention/
15  dashboard/
16  widgets/
17  main.dart

```

5.4 Communication REST

Les échanges suivent le principe requête/réponse HTTP. Flutter envoie une requête au backend, celui-ci vérifie l'authentification, exécute la logique métier puis renvoie une réponse JSON.

TABLE 5.1 – Exemples d’endpoints

Méthode	Endpoint	Fonction
POST	/api/auth/login	Authentification
GET	/api/tpe	Liste des TPE
GET	/api/tpe/id	Détail d’un TPE
POST	/api/tpe	Création d’un TPE
PUT	/api/tpe/id	Modification
DELETE	/api/tpe/id	Suppression
GET	/api/stations	Liste des stations
POST	/api/pannes	Déclaration d’une panne
POST	/api/interventions	Création d’une intervention

Chapitre 6

Mise en place de l'environnement

6.1 Préparation du poste

Le développement nécessite un environnement comprenant Java, un environnement de développement, MySQL, Flutter et les outils Android nécessaires aux tests de l'application mobile.

6.2 Projet Spring Boot

Le projet Spring Boot est initialisé avec les dépendances :

- Spring Web ;
- Spring Data JPA ;
- Spring Security ;
- MySQL Driver ;
- Validation.

La gestion JWT est ensuite ajoutée au projet.

6.3 Configuration MySQL

Un schéma de base de données est créé pour stocker les données du système.

Exemple de configuration de développement :

Listing 6.1 – Configuration indicative

```
1 spring.datasource.url=jdbc:mysql://localhost:3306/tpe_management
2 spring.datasource.username=root
3 spring.datasource.password=*****
4
```

```
5 spring.jpa.hibernate.ddl-auto=update
6 spring.jpa.show-sql=true
7
8 server.port=8080
```

Le paramètre `ddl-auto=update` est adapté au développement, mais une stratégie plus contrôlée doit être utilisée dans un contexte de production.

6.4 Initialisation Flutter

Le projet mobile est créé avec Flutter. Les packages utilisés ou prévus comprennent :

- `http` ou `dio` pour les appels réseau ;
- `provider` ou `riverpod` pour la gestion d'état ;
- `flutter_secure_storage` pour le token ;
- `fl_chart` pour les graphiques.

6.5 Premier test d'intégration

Avant de développer les fonctionnalités métier, un endpoint très simple tel que `GET /ping` peut être créé.

Listing 6.2 – Premier endpoint de test

```
1 @RestController
2 public class PingController {
3
4     @GetMapping("/ping")
5     public String ping() {
6         return "API OK";
7     }
8 }
```

Ce test valide la chaîne :

Flutter → HTTP → Spring Boot → Réponse

Chapitre 7

Développement du backend

7.1 Création des entités

Les entités JPA représentent les objets persistants du système. Une entité TPE peut être représentée de manière simplifiée ainsi :

Listing 7.1 – Exemple d’entité TPE

```
1 @Entity
2 public class Tpe {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @Column(unique = true, nullable = false)
9     private String numeroSerie;
10
11     @Enumerated(EnumType.STRING)
12     private EtatTpe etat;
13
14     @ManyToOne
15     private Station station;
16
17     // getters et setters
18 }
```

7.2 Repository

Spring Data JPA permet de réduire le code nécessaire à l'accès aux données.

Listing 7.2 – Repository TPE

```
1 public interface TpeRepository
2     extends JpaRepository<Tpe, Long> {
3
4     Optional<Tpe> findByNumeroSerie(String numeroSerie);
5
6     List<Tpe> findByEtat(EtatTpe etat);
7 }
```

7.3 Service

La couche service contient les règles métier. Cette séparation évite de placer toute la logique dans les contrôleurs.

Listing 7.3 – Structure du service

```
1 @Service
2 public class TpeService {
3
4     private final TpeRepository repository;
5
6     public TpeService(TpeRepository repository) {
7         this.repository = repository;
8     }
9
10    public List<Tpe> findAll() {
11        return repository.findAll();
12    }
13
14    public Tpe findById(Long id) {
15        return repository.findById(id)
16            .orElseThrow();
17    }
18 }
```

7.4 Contrôleur REST

Le contrôleur expose les opérations aux clients.

Listing 7.4 – Contrôleur REST simplifié

```
1 @RestController
2 @RequestMapping("/api/tpe")
3 public class TpeController {
4
5     private final TpeService service;
6
7     public TpeController(TpeService service) {
8         this.service = service;
9     }
10
11     @GetMapping
12     public List<Tpe> findAll() {
13         return service.findAll();
14     }
15
16     @GetMapping("/{id}")
17     public Tpe findById(@PathVariable Long id) {
18         return service.findById(id);
19     }
20 }
```

Chapitre 8

Authentification et sécurité

8.1 Objectif

La sécurité est une partie essentielle du système. Les données de maintenance ne doivent pas être accessibles sans authentification et certaines opérations doivent être réservées à l'administrateur.

8.2 JWT

JSON Web Token permet au serveur de délivrer un jeton après authentification. Le client conserve ce jeton et l'envoie dans les requêtes protégées.

Le schéma général est :

1. l'utilisateur saisit ses identifiants ;
2. Flutter envoie les informations au backend ;
3. le backend vérifie l'utilisateur ;
4. le backend génère un JWT ;
5. Flutter stocke le token de manière sécurisée ;
6. les requêtes suivantes utilisent le token.

8.3 Rôles

Deux rôles sont utilisés :

- ADMIN ;
- TECHNICIEN.

Les endpoints sensibles peuvent être protégés par rôle.

Listing 8.1 – Exemple indicatif de restriction

```
1 @PreAuthorize("hasRole('ADMIN')")
2 @PostMapping
3 public Tpe create(@RequestBody Tpe tpe) {
4     return service.create(tpe);
5 }
```

8.4 Bonnes pratiques

Les mots de passe ne doivent jamais être stockés en clair. Ils doivent être hachés avec un algorithme adapté. Le JWT doit avoir une durée de validité raisonnable et les clés secrètes ne doivent pas être directement publiées dans le code source.

Chapitre 9

Gestion du cycle de vie des TPE

9.1 Principe

Le cycle de vie constitue le coeur métier du projet. Chaque transition doit être déclenchée par une opération cohérente.

9.2 Affectation

Un TPE en stock peut être affecté à une station. Cette opération associe le terminal à la station et modifie son état.

9.3 Déclaration d'une panne

Lorsqu'un technicien constate un problème, il déclare une panne. Le système associe la panne au TPE et modifie l'état de ce dernier.

9.4 Réparation

Une intervention permet de suivre le traitement de la panne. Lorsque la réparation est terminée, le système peut passer le TPE à l'état *Réparé*.

9.5 Remise en service

Un TPE réparé peut être remis en service selon les règles définies. Lorsque le terminal est à nouveau affecté, son état devient *Affecté*.

9.6 Hors service

Si le terminal ne peut plus être utilisé, il est déclaré hors service. Cette transition doit être contrôlée car elle représente une décision importante sur le matériel.

9.7 Implémentation des transitions

Il est préférable de centraliser les transitions dans une méthode de service plutôt que de permettre à chaque contrôleur de modifier directement l'état.

Listing 9.1 – Exemple conceptuel

```
1 public void changerEtat(Tpe tpe, EtatTpe nouvelEtat) {  
2  
3     if (!transitionAutorisee(tpe.getEtat(), nouvelEtat)) {  
4         throw new IllegalStateException(  
5             "Transition non autorisee");  
6     }  
7  
8     tpe.setEtat(nouvelEtat);  
9     repository.save(tpe);  
10 }
```

Chapitre 10

Gestion des pannes et interventions

10.1 Déclaration d'une panne

Une panne contient les informations permettant de comprendre le problème rencontré. Le type de panne permet de standardiser les catégories utilisées par les techniciens.

10.2 Référentiel des types de panne

L'administrateur peut gérer un référentiel comprenant par exemple :

- problème matériel ;
- problème d'alimentation ;
- problème réseau ;
- problème d'écran ;
- problème de lecteur ;
- problème logiciel ;
- autre.

Ces valeurs sont données à titre d'exemple et peuvent être adaptées aux besoins réels.

10.3 Intervention

Une intervention représente le travail effectué par un technicien. Elle peut comprendre :

- type d'intervention ;
- date de début ;

- date de fin ;
- diagnostic ;
- action réalisée ;
- résultat ;
- technicien responsable.

10.4 Historique

L'historique permet de reconstituer les événements associés à un TPE. Il représente un élément important pour le diagnostic et le suivi de la qualité du parc.

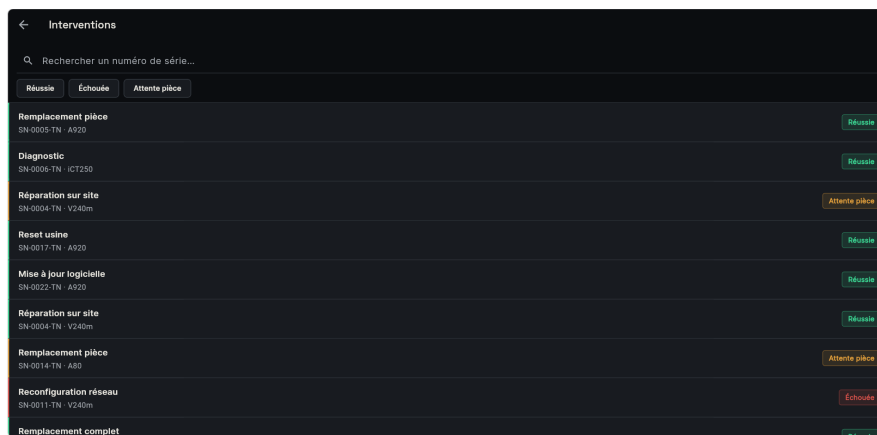


FIGURE 10.1 – Écran d'historique d'un TPE à insérer

Chapitre 11

Développement de l'application Flutter

11.1 Objectif de l'interface

L'application mobile doit permettre au technicien d'effectuer rapidement les opérations quotidiennes. L'interface doit donc être simple, lisible et organisée autour des principales fonctionnalités.

11.2 Écran de connexion

L'écran de connexion contient les champs nécessaires à l'authentification. Après une connexion réussie, l'utilisateur est redirigé vers l'écran principal.

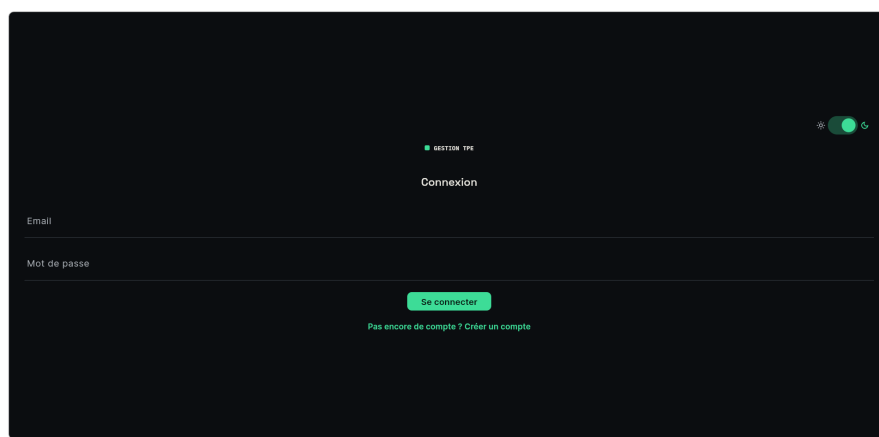


FIGURE 11.1 – Capture de l'écran de connexion Flutter

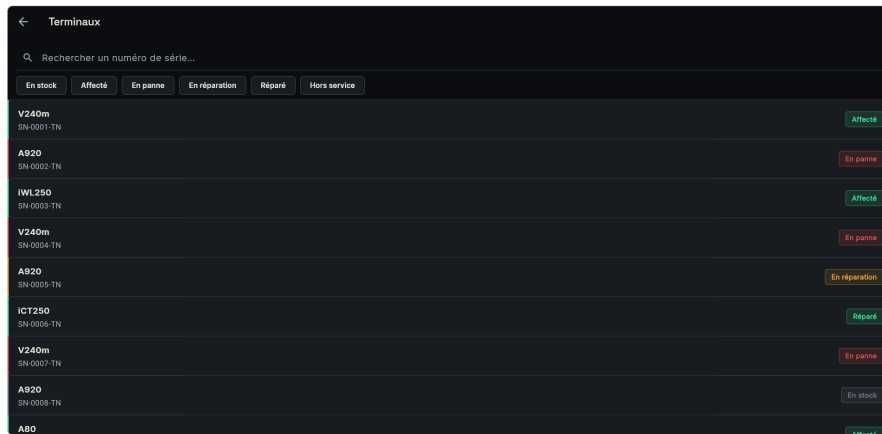


FIGURE 11.2 – Capture de la liste des TPE

11.3 Liste des TPE

La liste affiche les équipements avec les informations les plus utiles : numéro de série, état et station.

Une recherche peut être réalisée à partir du numéro de série.

11.4 Détail d'un TPE

L'écran de détail présente les informations du terminal ainsi que les actions disponibles selon son état et le rôle de l'utilisateur.

11.5 Ajout et modification

Les administrateurs peuvent accéder aux formulaires de création et de modification d'un TPE.

11.6 Gestion des stations

L'application permet de consulter les stations et leurs TPE affectés. L'administrateur peut créer ou modifier une station.

11.7 Pannes et interventions

Le technicien peut déclarer une panne depuis le détail du TPE. Une intervention peut ensuite être créée pour suivre la résolution.

Chapitre 12

Dashboard et statistiques

12.1 Objectif

Le tableau de bord permet d'obtenir une vision synthétique de l'état du parc. Il doit présenter les informations les plus utiles sans surcharger l'utilisateur.

12.2 Indicateurs

Les premiers indicateurs peuvent comprendre :

- nombre total de TPE ;
- TPE en stock ;
- TPE affectés ;
- TPE en panne ;
- TPE en réparation ;
- TPE hors service ;
- nombre d'interventions ;
- nombre de pannes sur une période donnée.

12.3 Statistiques avancées

Des statistiques supplémentaires peuvent identifier :

- les TPE les plus souvent en panne ;
- les stations avec le plus d'incidents ;
- le nombre d'interventions par mois ;
- la répartition des types de panne ;
- la durée moyenne de réparation.

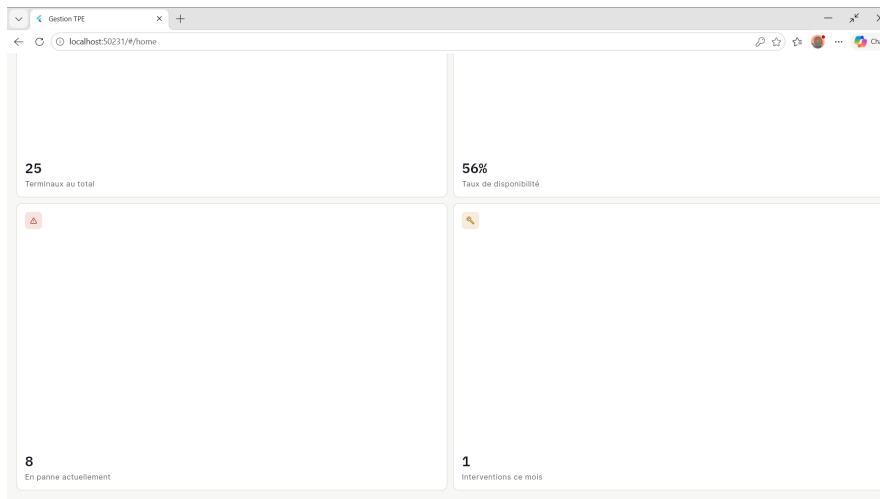


FIGURE 12.1 – Dashboard avec indicateurs et graphiques

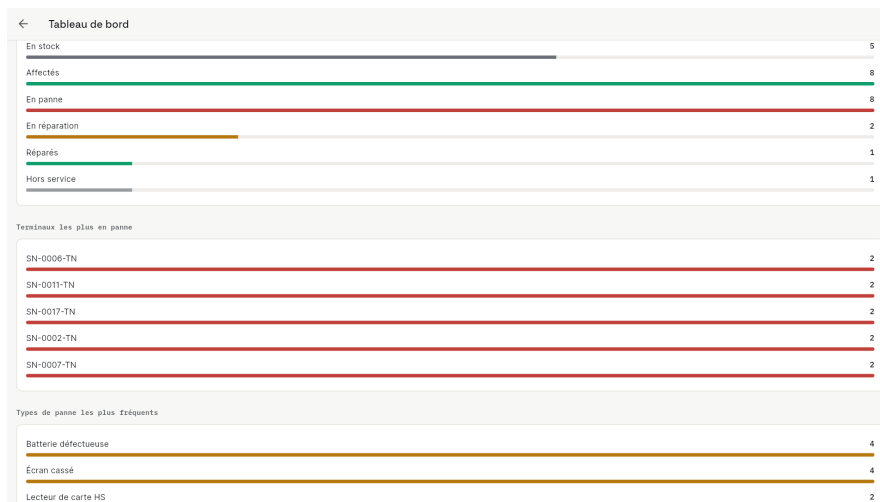


FIGURE 12.2 – stat

12.4 Graphiques Flutter

Le package `fl_chart` peut être utilisé pour représenter visuellement les données.

Chapitre 13

API REST et documentation

13.1 Principes REST

L'API est conçue autour des ressources métier. Les opérations principales utilisent les méthodes HTTP :

- GET pour consulter ;
- POST pour créer ;
- PUT pour modifier ;
- DELETE pour supprimer.

13.2 Format JSON

Les données échangées sont représentées en JSON.

Listing 13.1 – Exemple de réponse TPE

```
1 {  
2   "id": 1,  
3   "numeroSerie": "TPE-00125",  
4   "etat": "AFFECTE",  
5   "station": {  
6     "id": 3,  
7     "nom": "Station Centre"  
8   }  
9 }
```

13.3 Gestion des erreurs

L'API doit retourner des codes HTTP cohérents. Par exemple :

TABLE 13.1 – Codes HTTP utilisés

Code	Signification
200	Requête réussie
201	Ressource créée
400	Requête invalide
401	Non authentifié
403	Accès interdit
404	Ressource inexistante
500	Erreur serveur

13.4 Swagger / OpenAPI

La documentation OpenAPI permet de tester et de comprendre les endpoints. Elle facilite également la communication entre le développeur backend et le développeur mobile.

Chapitre 14

Tests et validation

14.1 Stratégie

Les tests sont réalisés à plusieurs niveaux :

1. tests des services critiques ;
2. tests des endpoints REST ;
3. tests manuels avec Postman ou Swagger ;
4. tests fonctionnels de l'application Flutter ;
5. tests de scénarios complets.

14.2 Scénario d'authentification

1. ouvrir l'application ;
2. saisir un compte valide ;
3. envoyer le formulaire ;
4. vérifier l'obtention du JWT ;
5. vérifier l'accès à l'écran principal.

14.3 Scénario de panne

1. rechercher un TPE affecté ;
2. ouvrir son détail ;
3. déclarer une panne ;
4. vérifier que l'état change ;
5. créer une intervention ;

6. terminer l'intervention ;
7. vérifier l'état final.

14.4 Tests unitaires

Les tests unitaires doivent se concentrer en priorité sur les règles métier critiques, notamment les transitions d'état.

TABLE 14.1 – Exemple de plan de tests

ID	Test	Résultat attendu
T01	Connexion valide	JWT retourné
T02	Connexion invalide	Accès refusé
T03	Création TPE	TPE enregistré
T04	Numéro de série dupliqué	Erreur de validation
T05	Déclaration panne	État modifié
T06	Transition interdite	Opération refusée
T07	Création intervention	Intervention créée
T08	Accès admin par technicien	Accès refusé

Chapitre 15

Déploiement

15.1 Architecture de déploiement

L'architecture finale peut être représentée par trois composants principaux : le smartphone ou émulateur, le serveur Spring Boot et le serveur MySQL.

15.2 Environnement de développement

Pendant le développement, Spring Boot peut fonctionner localement sur le port 8080 et MySQL sur son port standard. Flutter peut être exécuté sur un émulateur Android ou un appareil physique.

15.3 Environnement de production

Pour une mise en production, il serait possible d'utiliser un serveur Linux. L'API pourrait être exécutée comme service et la base de données hébergée sur un serveur dédié ou une infrastructure cloud.

15.4 Sécurité du déploiement

Les communications devraient utiliser HTTPS. Les secrets de connexion et les clés JWT doivent être stockés dans des variables d'environnement ou un gestionnaire de secrets.

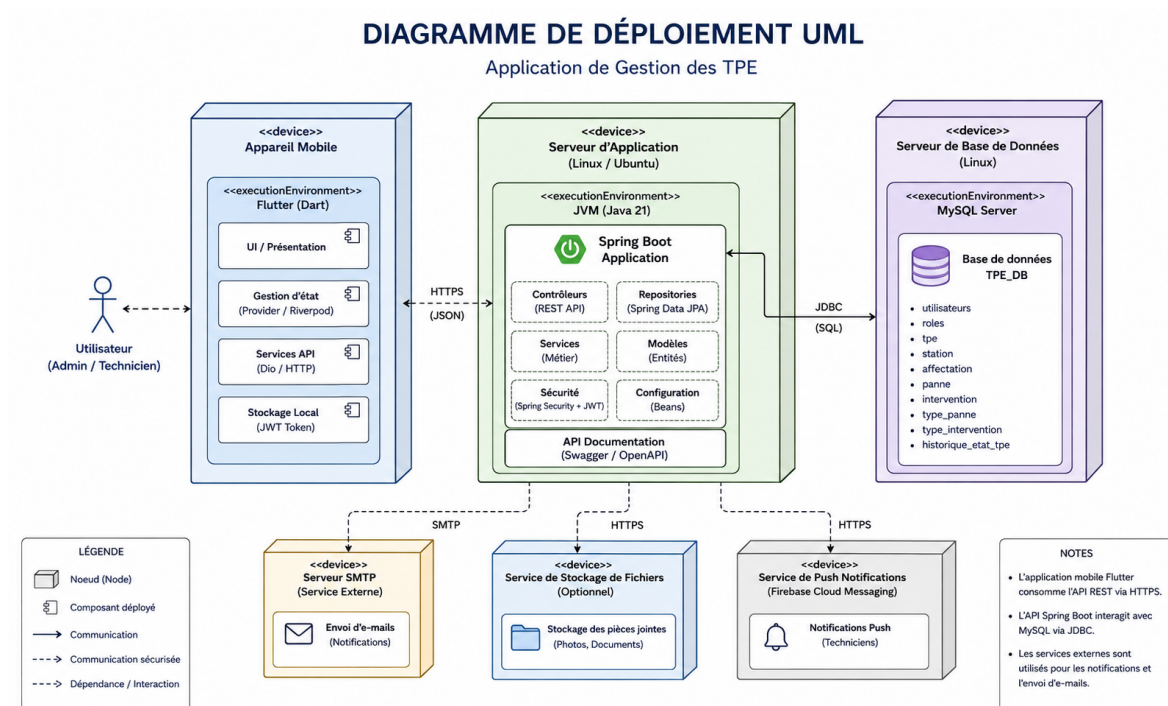


FIGURE 15.1 – Diagramme de déploiement UML

Chapitre 16

Organisation et planification

16.1 Semaine 1 : conception et setup

La première semaine est consacrée à la conception fonctionnelle, aux règles métier, au MCD/MLD et à l'installation de l'environnement.

16.2 Semaine 2 : coeur backend

La deuxième semaine porte sur l'authentification, les entités, les CRUD, les affectations, les référentiels et les pannes.

16.3 Semaine 3 : statistiques et Flutter

La troisième semaine permet de terminer le backend et de commencer les écrans mobiles principaux.

16.4 Semaine 4 : intégration et livraison

La dernière semaine est consacrée aux écrans restants, au dashboard, aux tests et à la documentation.

TABLE 16.1 – Planning prévisionnel

Période	Travail
S1	Analyse, UML, MCD/MLD, Spring Boot, Flutter, MySQL
S2	JWT, entités, CRUD, affectations, pannes, interventions
S3	Statistiques, Swagger, recherche, écrans TPE
S4	Stations, interventions, dashboard, tests, documentation

Chapitre 17

Difficultés et solutions

17.1 Complexité du cycle de vie

Le principal risque fonctionnel est de permettre des transitions incohérentes. La solution consiste à centraliser les transitions dans la couche service.

17.2 Communication mobile/backend

Les problèmes de communication peuvent provenir de l'adresse IP, du port, du réseau local ou de la configuration de l'émulateur. Une première route `/ping` permet de vérifier la connexion avant d'implémenter les fonctionnalités métier.

17.3 Authentification

La gestion du token JWT nécessite une bonne organisation du stockage et des requêtes. Un intercepteur HTTP peut automatiquement ajouter le header d'autorisation.

17.4 Gestion des données

Les relations JPA doivent être conçues avec attention afin d'éviter les cycles JSON ou le chargement excessif des relations. L'utilisation de DTO peut améliorer la séparation entre modèle interne et données exposées.

17.5 Gestion du temps

Si le temps devient insuffisant, les fonctionnalités les moins prioritaires sont les graphiques avancés, les tests exhaustifs et certaines opérations secondaires. En revanche,

l'authentification, le cycle de vie et les fonctionnalités principales de TPE doivent rester prioritaires.

Chapitre 18

Bilan et perspectives

18.1 Bilan

Le projet permet de mettre en pratique plusieurs compétences du génie logiciel. Il combine conception UML, base de données relationnelle, développement backend, développement mobile, sécurité et tests.

L'utilisation de Spring Boot permet de structurer l'API selon une architecture claire. Flutter permet de construire une interface mobile unique et évolutive. MySQL assure la persistance des informations.

18.2 Compétences acquises

Le projet permet notamment de développer les compétences suivantes :

- conception d'une application complète ;
- modélisation UML et relationnelle ;
- développement avec Spring Boot ;
- utilisation de JPA ;
- conception d'API REST ;
- sécurisation par JWT ;
- développement Flutter ;
- communication HTTP/JSON ;
- gestion des bases MySQL ;
- réalisation de tests.

18.3 Perspectives

Plusieurs améliorations peuvent être envisagées :

- notifications lors d'une panne ;
- scan du numéro de série par caméra ;
- export des statistiques en PDF ou Excel ;
- historique complet des changements d'état ;
- système de recherche plus avancé ;
- gestion de pièces de rechange ;
- système de notifications pour les interventions urgentes ;
- déploiement cloud ;
- supervision et journalisation avancées.

Conclusion générale

Ce projet a pour objectif de répondre à une problématique concrète : assurer une gestion structurée et sécurisée des terminaux de paiement électronique.

La solution proposée repose sur une séparation claire entre l'application mobile, l'API backend et la base de données. Cette architecture facilite la maintenance, l'évolution et les tests du système.

Le cycle de vie du TPE constitue le cœur de la logique métier. La gestion des états, des pannes et des interventions permet de suivre l'équipement depuis son stockage jusqu'à son affectation, sa maintenance et éventuellement sa mise hors service.

Le projet représente également une expérience complète de développement logiciel. Il mobilise des connaissances en analyse, conception, programmation, bases de données, sécurité et développement mobile.

Les fonctionnalités présentées peuvent être enrichies progressivement afin de transformer le prototype en une solution complète destinée à une utilisation réelle.

Annexe A

Annexe A – Arborescence Spring Boot

```
1 src/  
2   main/  
3     java/  
4       com.example.tpe/  
5         controller/  
6         service/  
7         repository/  
8         model/  
9         dto/  
10        security/  
11        exception/  
12        TpeApplication.java  
13    resources/  
14        application.properties
```

Annexe B

Annexe B – Arborescence Flutter

```
1 lib/  
2   core/  
3   models/  
4   services/  
5   providers/  
6   screens/  
7     login/  
8     tpe/  
9     station/  
10    panne/  
11    intervention/  
12    dashboard/  
13  widgets/  
14  main.dart
```

Annexe C

Annexe C – Exemple SQL

Listing C.1 – Création indicative de la base

```
1 CREATE DATABASE tpe_management;
2
3 USE tpe_management;
4
5 CREATE TABLE station (
6     id BIGINT PRIMARY KEY AUTO_INCREMENT,
7     nom VARCHAR(100) NOT NULL,
8     adresse VARCHAR(255)
9 );
10
11 CREATE TABLE tpe (
12     id BIGINT PRIMARY KEY AUTO_INCREMENT,
13     numero_serie VARCHAR(100) NOT NULL UNIQUE,
14     etat VARCHAR(50) NOT NULL,
15     station_id BIGINT,
16     CONSTRAINT fk_tpe_station
17         FOREIGN KEY (station_id) REFERENCES station(id)
18 );
```

Annexe D

Annexe D – Exemple de requêtes API

```
1 POST /api/auth/login
2
3 GET /api/tpe
4 GET /api/tpe/1
5 GET /api/tpe?numeroSerie=TPE-00125
6 POST /api/tpe
7 PUT /api/tpe/1
8 DELETE /api/tpe/1
9
10 GET /api/stations
11 POST /api/stations
12
13 POST /api/pannes
14 GET /api/tpe/1/historique
15
16 POST /api/interventions
17 GET /api/statistiques
```

Annexe E

Annexe E – Checklist de livraison

- ☐ Projet Spring Boot compilable.
- ☐ Base MySQL configurée.
- ☐ API REST fonctionnelle.
- ☐ Authentification JWT fonctionnelle.
- ☐ Gestion des rôles.
- ☐ CRUD TPE.
- ☐ Gestion des stations.
- ☐ Affectation des TPE.
- ☐ Gestion des pannes.
- ☐ Gestion des interventions.
- ☐ Cycle de vie contrôlé.
- ☐ Application Flutter fonctionnelle.
- ☐ Dashboard.
- ☐ Tests réalisés.
- ☐ Documentation Swagger.
- ☐ README finalisé.